

INTERNLOOM HACKATHON 2026

BMS College of Engineering, Bangalore • July 18, 2026

TRACK 2 — AI DEVELOPMENT

Problem Statement

Build the InternLoom Resume Shortlisting Engine

FORMAT Individual	DURATION 4–5 Hours	AI TOOLS Fully Allowed	TOTAL MARKS 100 Points
-----------------------------	------------------------------	----------------------------------	----------------------------------

1. Context & Background

InternLoom receives hundreds of student applications for every job listing it hosts. Each applicant submits a resume — a PDF document with no guaranteed structure, no consistent format, and no standard field layout. One candidate uses a two-column Canva template. Another submits a plain text document. A third has a scanned image of a printed page. The companies on the other side have Job Descriptions with specific skill requirements, CGPA expectations, and preferred experience signals.

The gap between them — figuring out which PDF resumes actually match which Job Description — is today done manually by InternLoom's team. It is slow, inconsistent, and does not scale.

You are being asked to build the AI system that replaces that manual step: a Resume Shortlisting Engine that accepts a folder of real PDF resumes and a set of Job Descriptions, and produces a ranked, scored, explainable shortlist — automatically.

This is a two-stage engineering problem. Stage 1 is extraction: turning unstructured PDF documents into structured, queryable data. Stage 2 is matching: ranking and scoring that structured data against JD requirements. Both stages are hard. Most candidates will underestimate Stage 1.

⚠ Important

⚠ This problem has no single correct solution. The resumes are real, unformatted PDFs with inconsistent layouts. Your parsing and matching logic will be tested against data it has never seen. What judges penalise is a system that only works on well-formatted inputs and breaks on anything unusual — because that is exactly what a real hiring pipeline encounters every day.

2. What You Receive at the Start

2.1 Resume Dataset — PDF Files via Google Drive

At 10:00 AM when the build phase begins, you will be given access to a Google Drive folder containing real student resume PDFs. These are the raw inputs your engine must process.

📁 RESUME DATASET — GOOGLE DRIVE LINK

<https://drive.google.com/drive/folders/1G76vUJ1IRAWxltZQeMmvTX5mltUealcW>

This link will be distributed at 10:00 AM when the build phase begins. Access is read-only. Do not attempt to add or delete files.

Dataset Reality Check

The PDFs are real resumes with real variation. You will encounter: single-column and multi-column layouts, Canva-designed templates with text in floating boxes, resumes with tables, resumes with no

section headers, resumes where skills are listed as icons or graphics rather than text, and at least one scanned image resume where standard text extraction will return nothing.

Your engine must handle this variation gracefully. A parser that works on 20 out of 30 resumes and silently fails on the rest is not acceptable. Failed parses must be flagged explicitly in your output.

2.2 Job Descriptions — 5 Roles

Five Job Descriptions are provided in this document (Section 6). Each JD specifies required skills, preferred skills, minimum CGPA, and number of available slots. Your engine must be able to process any JD — not just these five. Judges may test it with a sixth JD not listed here during your demo.

2.3 Expected Output Format

For each JD, your engine must produce:

- A ranked shortlist of candidates in descending order of match score
- A score out of 100 for each candidate
- A confidence level: High / Medium / Low — your assessment of how certain the score is, based on how cleanly the resume parsed
- Three bullet points of reasoning per candidate: top reasons for their ranking (positive or negative)
- A parse quality flag per candidate: Clean / Partial / Failed — how successfully the PDF was extracted
- A summary line per JD: candidates evaluated, candidates shortlisted, score cutoff used, and number of parse failures

3. Stage 1 — PDF Parsing & Data Extraction

Before any matching can happen, your engine must convert each PDF resume into a structured object. This is the hardest part of the problem and the part most candidates will attempt to skip or oversimplify.

3.1 What You Must Extract

For each resume PDF, your engine must attempt to extract the following fields. Not every resume will have all fields — missing fields must be recorded as null, not assumed.

- Full name
- Email address
- Phone number (optional — not used for scoring but extracted if present)
- College / University name
- Degree and branch (e.g., B.E. Computer Science)
- Graduation year

- CGPA or percentage — note that some resumes show percentage, others show CGPA on a 10-point scale, others show GPA on a 4-point scale. Your engine must normalise all three to a 10-point CGPA equivalent.
- Skills — as a list, not a paragraph
- Projects — title and one-line description for each
- Internship or work experience — company, role, duration
- Certifications or courses (if present)

⚠ Tricky Part 1

Tricky Part 1 — PDF Layout Chaos

Standard PDF text extraction libraries (pdfplumber, PyMuPDF, pdfminer) extract text in reading order — which is left-to-right, top-to-bottom. This works on single-column resumes. It breaks on two-column layouts, where the extracted text interleaves content from both columns in unpredictable ways.

Example: A two-column resume with Education on the left and Skills on the right may extract as: "B.E. Computer SciencePython JavaScript React" with no separator between the two columns.

Your parser must detect and handle multi-column layouts. Approaches include: bounding box analysis (PyMuPDF supports this), splitting the page into left/right halves and extracting separately, or using an LLM to interpret raw extracted text. Your Design Decisions document must describe which approach you used and what its known failure cases are.

⚠ Tricky Part 2

Tricky Part 2 — The Scanned Resume

At least one resume in the dataset is a scanned image — a photo or scan of a printed page saved as a PDF. Standard text extraction on this file returns empty or near-empty text because there are no text layers, only image pixels.

Your engine must detect when a PDF yields no extractable text and respond in one of two ways: (a) apply OCR using a library such as pytesseract or easyocr to attempt text recovery, or (b) flag the resume as Failed Parse with a clear explanation in the output. Option (a) earns more marks. Option (b) is acceptable if documented.

Judges will specifically ask about this resume during your demo. 'I didn't know it was there' is not an acceptable answer.

3.2 Grade Normalisation

Resumes in India express academic performance in at least three different formats. Your engine must handle all three and convert to a unified 10-point CGPA scale before scoring:

- CGPA on 10-point scale — use as-is (e.g., 8.4 / 10)
- Percentage — divide by 9.5 to approximate CGPA (e.g., 79% → 8.3)
- GPA on 4-point scale — multiply by 2.5 to approximate CGPA (e.g., 3.6 / 4 → 9.0)

If the scale is ambiguous (e.g., a number listed without context), your engine must make an assumption, state it in the output, and flag it as Low confidence.

4. Stage 2 — Scoring & Matching Engine

Once resumes are parsed into structured objects, your engine scores each candidate against each JD. The matching logic must account for the following dimensions. How you weight them is your design decision — but you must document and defend your choices.

4.1 Skill Match — Required vs Preferred

The JD separates required skills from preferred skills. These carry different weight. Your scoring must treat them differently.

The harder problem is that skills on resumes are rarely stated exactly as in the JD. Your engine must handle:

- Exact match: JD says "React.js", resume says "React.js" → full credit
- Synonym match: JD says "React.js", resume says "ReactJS" or "React" → must be treated as a match
- Partial match: JD says "PostgreSQL", resume says "SQL" or "databases" → partial credit, flagged
- Implicit match: Resume shows a project that clearly used REST APIs but never uses that exact term → inferred match with Low confidence

⚠ Tricky Part 3

Tricky Part 3 — Skill Extraction from Unstructured Text

Many resumes do not have a clearly labelled "Skills" section. Skills appear inline in project descriptions, in certification titles, or embedded in experience descriptions. A parser that only looks for a section header labelled "Skills" or "Technical Skills" will miss these candidates entirely.

Your engine must extract skills from the full resume text, not just from a dedicated section. This requires either: a predefined skills vocabulary that you search for across all text (fast but rigid), an NLP model that identifies technology names contextually (more flexible but requires more setup), or an LLM prompt that extracts skills from the full text (most flexible, but prompt design matters).

Document which approach you chose and what it would miss.

4.2 Signal Conflict Resolution

Parsed resumes will present conflicting signals. Your engine cannot ignore them — it must resolve them with documented logic:

- High CGPA, zero projects — strong academic signal, weak practical signal

- Low CGPA, multiple deployed projects — weak academic signal, strong practical signal
- Perfect skill match, non-CS degree — role fit conflicts with educational background
- Partial parse — strong what was extracted, but 40% of the resume failed to parse. Is a partial high-scorer better or worse than a complete medium-scorer?

⚠ Tricky Part 4

Tricky Part 4 — Parse Quality Affects Score Confidence

A resume that only partially parsed gives your engine incomplete signal. If you score a candidate based on 60% of their resume, your score is meaningless for the 40% you missed. Simply scoring the available data and presenting it as a reliable result is a product failure.

Your engine must factor parse quality into confidence level. A Partial parse must produce a Low or Medium confidence score at most, regardless of how strong the extracted data looks. A Failed parse should not produce a score at all — it should produce a flag and a recommendation for human review.

Document this logic explicitly. Judges will look for it.

4.3 Slot Awareness

Each JD specifies how many slots are available. Your shortlist must respect this. If 12 candidates score above your threshold for a role with 8 slots, you must shortlist 8 — not 12. Candidates ranked 9th and below must appear in a separate "Reserve" list with their scores, not be silently excluded.

5. Technical Requirements

5.1 Stack & Libraries

- Language: Python only.
- PDF extraction: pdfplumber, PyMuPDF (fitz), pdfminer.six — any or all permitted. Document which you used for which type of PDF and why.
- OCR (if attempted): pytesseract, easyocr, or equivalent.
- NLP / AI: spaCy, HuggingFace, OpenAI API, Anthropic API, LangChain, sentence-transformers — all permitted. Document API usage in AI log.
- Interface: At minimum, a CLI that accepts the Drive folder path (or a local copy of it) and outputs the shortlist. A web UI is a bonus (see Section 7).

5.2 What You Must Submit

- **GitHub repository — public or shared with judges. README must cover setup and a working example run in under 5 minutes.**
- **requirements.txt — with version pins. Judges must be able to install and run without hunting for dependencies.**

- **Sample output file** — run your engine on the provided Drive folder and include the output in your repo. This is your baseline output that judges compare against their own evaluation.
- **Parse quality report** — a separate file listing every resume, its parse status (Clean / Partial / Failed), and what failed if applicable. This is evaluated independently.
- **AI Usage Log** — max 200 words. What tools you used, for which specific task, and what you changed or built beyond what they generated.
- **Design Decisions Document** — markdown or text file answering all four Tricky Parts. One paragraph each describing your actual implementation, not theory.

5.3 Hard Constraints — Instant Disqualification

Disqualification Criteria

The following will result in immediate disqualification:

- Engine only works on pre-processed or manually cleaned input. It must accept the raw PDF files from the Drive link without any manual preprocessing step.
- Failed parses are silently ignored. Every resume must appear in the output — either with a score or with a clear Failed Parse flag.
- Hardcoded results. The engine must derive all outputs from the algorithm, not from pre-set candidate scores.
- No parse quality report. With real PDFs as input, parse transparency is mandatory.
- No Design Decisions document. Four explicit tricky parts, all of which require implementation decisions. A missing doc means judges cannot evaluate architectural thinking.

6. Job Descriptions

The following five JDs represent the roles InternLoom is sourcing for Relyntis (UTS). Total slots: 30. Your engine must be able to accept any JD — judges may introduce a sixth during the demo.

Role	Required Skills	Preferred Skills	CGPA Min	Slots
Frontend Developer	HTML, CSS, JavaScript React.js or Next.js Git/GitHub REST API consumption	TypeScript Redux / Zustand Responsive design Unit testing (Jest)	6.5 or above	8
Backend Developer	Node.js / Python / Java REST API design SQL or NoSQL DB Git/GitHub	Docker basics Auth (JWT/OAuth) API documentation Cloud basics	6.5 or above	10
Full Stack Developer	React.js + Node.js or equivalent Database (any) REST APIs (build + consume)	Full deployed project Firebase or cloud DB CI/CD awareness GraphQL	7.0 or above	7

	Git/GitHub, deployment basics			
Database Developer	MySQL or PostgreSQL Schema design & normalisation Query writing & optimisation Basic indexing	MongoDB / Redis Stored procedures Aggregation pipelines ERD design	6.0 or above	3
API Integration Developer	REST API consumption Postman / Swagger JSON handling Any backend language	OAuth / API key auth Webhook handling Third-party SDK integration API documentation writing	6.0 or above	2

7. Bonus Challenges

Attempt only after your core engine is working and producing output on all resumes. Each bonus is independent.

Bonus A — Web Interface (+10 marks)

Build a simple web UI (Flask, FastAPI + HTML, or Streamlit) where a recruiter can: upload a folder of PDF resumes or a single PDF, paste a JD, and receive the shortlist as an interactive table with scores, confidence levels, parse quality flags, and reasoning visible per candidate. Export to CSV must work.

Bonus B — OCR Fallback (+10 marks)

Implement automatic OCR fallback for resumes that yield less than 50 words of extractable text. The engine must detect low-yield PDFs, apply OCR automatically, and attempt scoring from the OCR output with appropriate confidence flags. Demonstrate this on the scanned resume in the dataset during the demo.

Bonus C — Live JD Parsing (+7 marks)

Accept a raw, unstructured job description in plain English (the kind pasted from a LinkedIn posting) and parse it to extract required skills, preferred skills, and CGPA expectations before scoring. Demonstrate this with one unstructured JD during the demo.

Bonus D — Score Stability (+8 marks)

Demonstrate that your engine produces consistent scores across three runs on the same input. If scores vary by more than 5 points between runs (common with LLM-based approaches), document why and implement a determinism layer. Submit a test script that runs the engine three times and compares outputs programmatically.

8. Evaluation Rubric

Evaluation Criterion	Max Marks	What Judges Look For
PDF Parsing Quality	20	Does the engine handle multi-column layouts? Is the scanned resume detected and handled? Is parse quality reported per resume? Are all resumes present in output?
Data Extraction Accuracy	15	Are skills, CGPA, projects, and education extracted correctly from well-formed PDFs? Is grade normalisation implemented?
Scoring Algorithm Design	15	Is weighting documented? Required vs preferred treated differently? Parse quality factored into confidence? Slot awareness respected?
Skill Matching Quality	15	Synonyms handled? Skills extracted from non-labelled sections? Partial/implicit matches flagged appropriately?
Explainability of Output	15	Reasoning per candidate meaningful? Parse quality flags present? Reserve list produced for over-threshold candidates?
Code Quality & Structure	10	Parsing, scoring, and output layers separated? Readable? Handles exceptions without crashing?
Submission Completeness	10	README, requirements.txt, sample output, parse report, AI log, Design Decisions doc — all present and usable?

Total: 100 marks. Bonus marks (up to 35) are added on top.

9. Demo & Day Timeline

Time	Activity
9:00 – 9:30 AM	Registration. Verify Python environment, pip installs, and API keys. Download the Drive folder locally if your internet is reliable — do not depend on Drive access during the build phase.
9:30 – 10:00 AM	Opening keynote and chief guest addresses. Problem statement and Drive link distributed at 9:55 AM.

10:00 AM	🕒 BUILD PHASE BEGINS. Drive link is live. Download resumes immediately — do not wait.
10:00 – 11:00 AM	Recommended: spend the first hour entirely on PDF parsing. Get clean extraction working before touching the matching algorithm.
12:30 – 1:00 PM	Lunch break. Commits during this window will be flagged.
3:00 PM	🕒 BUILD PHASE ENDS. Final commit pushed. Include sample output and parse quality report in the repo.
3:00 – 3:30 PM	Submit parse quality report, Design Decisions doc, and AI Usage Log.
3:30 – 5:00 PM	Demos. Top 10 candidates demo — 8 minutes each. Judges will run your engine live on one unseen resume PDF.
5:00 PM	Results announced. Top 2 AI Dev performers considered for InternLoom AI Dev internship.

During the 8-minute demo, judges will: drop one unseen PDF resume into your engine and watch it parse and score live, ask 'Which resume in the dataset gave your parser the most trouble and why?', and ask 'What would your engine get wrong on a resume it has never seen?' These two questions have no right answer. They have a right way of answering.

Build something that reads between the lines.

internloom.com | [@Internloom](https://twitter.com/Internloom)